

Übung 3: Streams

1. Szenario

Wir haben eine Datenbank mit Filmbewertungen (`MovieRating`). Deine Aufgabe ist es, verschiedene Analysen und Abfragen auf diesen Daten durchzuführen. Dazu sollst du die **Java Streams API** nutzen, um die Daten effizient und deklarativ zu verarbeiten.

2. Vorbereitung

Erstelle (oder kopiere) die folgenden Basis-Klassen, die für die Aufgabe benötigt werden.

Datenmodell: `MovieRating.java`

```
import java.time.Duration;
import java.util.Set;

public record MovieRating(
    int id,
    String name,
    Float value,
    Integer votes,
    Duration runtime,
    Set<String> genres,
    String description
) {}
```

Datenquelle: `MovieRatingsDataset.java`

```
import java.util.stream.Stream;

public interface MovieRatingsDataset {
    Stream<MovieRating> load();
}
```

3. Aufgaben

Kopiere die Klasse `MovieRatingsService`. Implementiere alle Methoden, die aktuell `return null`; zurückgeben, unter Verwendung der Streams API.

Achte auf die Javadoc-Beschreibungen!

```
import java.time.Duration;
import java.util.Comparator;
import java.util.Optional;
import java.util.Set;
import java.util.stream.Stream;

public class MovieRatingsService {

    private final MovieRatingsDataset ds;

    public MovieRatingsService(MovieRatingsDataset ds) {
        this.ds = ds;
    }

    /**
```

```

    * Gibt einen Stream von Bewertungen zurück, begrenzt auf 'limit' Einträge.
    */
    public Stream<MovieRating> getRatings(long limit) {
        return null; // TODO
    }

    /**
     * Sortiert die Bewertungen nach Anzahl der Stimmen (votes).
     * @param ascending true für aufsteigend, false für absteigend.
     */
    public Stream<MovieRating> getRatingsSortedByVotes(boolean ascending) {
        return null; // TODO
    }

    /**
     * Filtert Bewertungen, die größer oder gleich dem Grenzwert (cutoff) sind.
     */
    public Stream<MovieRating> getRatingsGreaterThanOrEqualTo(double cutoffInclusive) {
        return null; // TODO
    }

    /**
     * Gibt die Namen der Filme alphabetisch sortiert zurück.
     * Rückgabetypp ist Stream<String>!
     */
    public Stream<String> getNamesSortedAlphabetically() {
        return null; // TODO
    }

    /**
     * Gibt alle vorkommenden Genres zurück (ohne Duplikate).
     */
    public Stream<String> getDistinctGenres() {
        return null; // TODO
    }

    /**
     * Gibt Filme zurück, die ALLE angegebenen Genres besitzen.
     */
    public Stream<MovieRating> getMoviesHavingAllGenres(Set<String> genre) {
        return null; // TODO
    }

    /**
     * Berechnet die Summe aller abgegebenen Stimmen (votes) über alle Filme.
     */
    public Long getTotalVoteCountOverAllRatings() {
        return null; // TODO
    }

    /**
     * Sortiert die Filme nach Laufzeit (runtime).
     */
    public Stream<MovieRating> getMovieRatingsSortedByDuration(boolean ascending) {
        return null; // TODO
    }
}

```

```

/**
 * Berechnet die Durchschnittsbewertung (value) aller Filme eines bestimmten Genres.
 * Tipp: mapToDouble().average()
 */
public Float getAverageRatingOfMoviesHavingGenre(String genre) {
    return null; // TODO
}

/**
 * Sucht einen Film anhand seines Namens.
 * Sollte ein Optional zurückgeben (leer, falls nicht gefunden).
 */
public Optional<MovieRating> findMovieWithName(String name) {
    return null; // TODO
}

/**
 * Gibt Filme zurück, deren Laufzeit KÜRZER ist als die angegebene Duration.
 */
public Stream<MovieRating> getMoviesWithDurationLessThan(Duration duration) {
    return null; // TODO
}
}

```

4. Bonusaufgabe (Experten)

Ergänze die Klasse `MovieRatingsService` um eine generische Methode `getRatingsSortedByField`. Diese soll es erlauben, dynamisch nach jedem Feld in `MovieRating` zu sortieren und Streams zu verketteten.

Gewünschte Verwendung:

```

service.getRatingsSortedByField(MovieRating::name, true)    // erst nach Name sortieren
        .getRatingsSortedByField(MovieRating::rating, false) // dann nach Rating absteigend
        .forEach(System.out::println);

```

Hinweis: Da `Stream` einmal konsumiert wird, ist die Verkettung oben im Beispiel pseudo-code-mäßig zu verstehen bzw. erfordert, dass `getRatingsSortedByField` wieder einen `Stream` liefert, auf dem man weiterarbeiten kann (was aber die vorherige Sortierung zunichte machen würde, wenn man nicht aufpasst, oder man implementiert es als `Comparator-Chain`). **Einfachere Variante:** Schreibe die Methode so, dass sie einen `Stream` zurückgibt, der nach *einem* Feld sortiert ist.